Code-Bausteine zum Ausschneiden

Je Algorithmus ein Kartensatz · Partnerarbeit: Bausteine legen, Einrückung durch Verschieben nach rechts zeigen

Einsatz zu Aufgabe 6 der Arbeitsblätter: Die Karten werden in die richtige Reihenfolge gelegt; jede Einrückungsstufe rückt die Karte um eine Kartenbreite-Viertel nach rechts. Danach wird der Bauplan auf dem Blatt ausgefüllt. Die Buchstaben stimmen mit dem Blatt überein.

Lineare Suche — `lineare_suche(liste, gesucht)`

A

```
if liste[i] == gesucht:
```

Treffer?

LINEARE SUCHE

B

```
return -1
```

Liste zu Ende: nicht gefunden

LINEARE SUCHE

C

```
def lineare_suche(liste, gesucht):
```

Funktion mit zwei Eingaben: die Liste und der gesuchte Wert

LINEARE SUCHE

D

```
return i
```

gefunden: Index zurückgeben, die Suche endet sofort

LINEARE SUCHE

E

```
for i in range(len(liste)):
```

i läuft über alle Indizes: 0, 1, 2, …

LINEARE SUCHE

Binäre Suche — `binaere_suche(liste, gesucht)`

A

```
if liste[mitte] < gesucht:
    links = mitte + 1
```

Mitte zu klein: rechts weitersuchen

BINÄRE SUCHE

B

```
def binaere_suche(liste, gesucht):
```

Funktion: sortierte Liste und gesuchter Wert

BINÄRE SUCHE

C

```
return -1
```

Suchbereich leer: nicht gefunden

BINÄRE SUCHE

D

```
mitte = (links + rechts) // 2
```

Mitte berechnen, abgerundet

BINÄRE SUCHE

E

```
links = 0
rechts = len(liste) - 1
```

Suchbereich am Anfang: die ganze Liste

BINÄRE SUCHE

F

```
else:
    rechts = mitte - 1
```

Mitte zu groß: links weitersuchen

BINÄRE SUCHE

G

```
while links <= rechts:
```

solange der Suchbereich nicht leer ist

BINÄRE SUCHE

H

```
if liste[mitte] == gesucht:
    return mitte
```

Treffer: Index zurückgeben

BINÄRE SUCHE

Bubble Sort — `bubble_sort(liste)`

A

```
if liste[i] > liste[i + 1]:
```

linker Nachbar größer?

BUBBLE SORT

B

```
return liste
```

fertig sortierte Liste zurückgeben

BUBBLE SORT

C

```
n = len(liste)
```

Anzahl der Elemente

BUBBLE SORT

D

```
merker = liste[i]
liste[i] = liste[i + 1]
liste[i + 1] = merker
```

tauschen mit einer Hilfsvariablen

BUBBLE SORT

E

```
def bubble_sort(liste):
```

Funktion: die Liste, die sortiert werden soll

BUBBLE SORT

F

```
for i in range(n - 1 - durchgang):
```

alle Nachbarpaare; hinten ist schon sortiert

BUBBLE SORT

G

```
for durchgang in range(n - 1):
```

höchstens n − 1 Durchgänge

BUBBLE SORT

Selection Sort — `selection_sort(liste)`

A

```
kleinster = start
```

Index des bisher kleinsten Werts

SELECTION SORT

B

```
return liste
```

fertig sortierte Liste zurückgeben

SELECTION SORT

C

```
def selection_sort(liste):
```

Funktion: die Liste, die sortiert werden soll

SELECTION SORT

D

```
if liste[i] < liste[kleinster]:
    kleinster = i
```

noch kleiner gefunden: Index merken

SELECTION SORT

E

```
for start in range(n - 1):
```

start = erster Platz des unsortierten Teils

SELECTION SORT

F

```
merker = liste[start]
liste[start] = liste[kleinster]
liste[kleinster] = merker
```

kleinsten Wert nach vorn tauschen

SELECTION SORT

G

```
n = len(liste)
```

Anzahl der Elemente

SELECTION SORT

H

```
for i in range(start + 1, n):
```

den Rest des unsortierten Teils durchsuchen

SELECTION SORT